Kata: Global Economic Risk Monitor

A hands-on exercise for data scientists & engineers

Target effort: 6–8 hours

The Brief

Your firm's Chief Economist wants a prototype: an application where analysts can visually assess **recession risk** and at least one other macro risk dimension (**inflation** or **financial volatility**) — with a geographic view and forward-looking model outputs.

It should feel like a product, not a notebook. Build it using an AI-assisted IDE (Claude Code, Cursor, Copilot, etc.).

Requirements

1. Data
Ingest macroeconomic and/or financial data from public sources into a structured store. You choose the sources and the schema. The pipeline should be re-runnable. Cover at least the **United States** — additional economies earn credit but aren't required.

2. Two models, both probabilistic
Build at least **two analytical models** covering different risk domains. One must address **recession risk**. The second can target inflation, volatility, or another macro risk you find compelling. Both should produce **probability estimates or regime classifications** — not just point forecasts or line charts. Be ready to explain your modeling choices.

3. A frontend with a map
Build a UI using React, Shiny, Streamlit (with real customization), or a comparable framework. It must include:
- An **interactive map** (choropleth or marker-based) that communicates risk geographically
- At least one **time-series visualization** with a model output overlay (forecast, probability, or regime bands)
- A way to **drill into detail** — clicking a region/country should reveal more than the overview shows

4. It runs
Someone else should be able to start your application from a single command and a short README. Containerization is a plus but not required.

Architecture Guidance

You have two paths — pick whichever lets you ship a better product in the time box:

- **Separated stack** (preferred): A Python/R backend serving an API, consumed by a frontend. This is closer to production and lets you demonstrate API design — but it's more wiring.
- **Single-process app**: A Shiny, Streamlit, or Dash application with models and data access built in. Faster to ship, but show us clean internal separation (data layer, model layer, presentation layer — not everything in one file).

Either path is acceptable. The separated stack will score higher on systems design if executed well, but a polished single-process app beats a half-wired API with a broken frontend.

Tips

- **Get data flowing first.** Don't touch a model until you can query your data store and see numbers. This is the #1 time trap.

- **The yield curve is not the only recession signal.** It's the most famous, but if your model is just a threshold on the 10Y-2Y spread, you haven't shown us much. We want to see that you understand the broader indicator landscape.

- **Probabilistic means something specific.** If your model outputs 0.73, you should be able to tell us: a probability of what, over what horizon, calibrated how, and with what uncertainty.

- **The map is a design problem.** A choropleth needs a single scalar per geography. How you compress your model outputs into a mappable risk score is a real decision — make it intentionally.

- **Budget time for the UI.** A working model behind an unreadable interface is a failed prototype. Allocate at least 1.5-2 hours to frontend polish.

- **Write a `DECISIONS.md`.** A short file (even a dozen bullet points) documenting "I chose X over Y because Z" is worth more to us than an extra feature. We're evaluating your judgment, not your output volume.

- **Commit as you go.** The git history is part of the deliverable. It shows how you think and sequence work.

Evaluation Criteria

Dimension	What we're assessing
Data engineering	Source selection, pipeline design, storage choices
Modeling rigor	Appropriate methods, probabilistic outputs, ability to explain
Frontend & GIS	Map works, charts are readable, feels like a product
Code quality	Clean structure, readability, separation of concerns
Decision-making	Trade-off awareness, `DECISIONS.md`, pragmatism under constraint
AI-assisted workflow	How you directed the AI, what you corrected, what you owned

We weight **judgment and craft** over feature count.

Deliverables

1. A Git repository containing the application
2. A README with setup instructions
3. A `DECISIONS.md` documenting your key choices and trade-offs
4. Be ready for a **20-minute walkthrough** — we'll run the app, ask you to explain your models, and dig into a section of code

---

Stretch (only after the core works)

- Add a second or third economy to the map with its own model outputs
- A third risk model (inflation regimes, volatility forecasting, contagion)
- Backtesting overlay: model predictions vs. realized recessions
- Containerize the full stack with Docker Compose
- LLM-generated natural-language risk summary on the dashboard

---

Good luck. Build something you'd want to use.